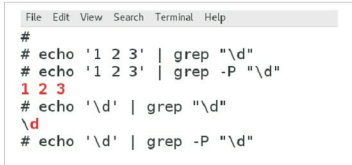


d and not a digit. But if the utility *grep* is using the Perl-style regular expressions then a digit will be matched by the same regular expression. In this series, all the regular expressions, and the strings and results obtained by using them are italicised to highlight them from normal text.

Figure 1 shows the output obtained when the default mode and the Perl style regular expressions are used with *grep*. The texts highlighted in red are the portions of the string matched by the given regular expression. In the figure, you can observe that the same regular expression while processing the same text in two different modes, matches different patterns.



```
File Edit View Search Terminal Help
#
# echo '1 2 3' | grep "\\d"
# echo '1 2 3' | grep -P "\\d"
1 2 3
# echo '\\d' | grep "\\d"
\\d
# echo '\\d' | grep -P "\\d"
```

Figure 1: Two regular expression styles in *grep*

As a side note, I would like to point out that all pattern matching utilities called *grep* are not the same. There are minor differences between the different implementations of *grep*. For example, all the implementations by GNU, IBM AIX and Solaris differ at least on certain functionalities. There are also variants of *grep* like *egrep*, *fgrep*, etc, which differ from *grep* in functionality as well as syntax.

Regular expressions in Python

Python is a general-purpose programming language invented by Guido van Rossum. The two active versions of it are Python 2 and Python 3, with Python 2.7 most likely being the last version of Python 2 and Python 3.6 being the current stable version of Python 3. Since we are concentrating on regular expressions in Python, we don't need to worry too much about the general differences between these two versions. Since both Python 2 and Python 3 use the same module for handling regular expressions, there is no real difference between the two. I have executed all the scripts in this article with Python 2.7.12. The Python module that supports regular expressions is called *re*. The module *re* supports Perl-style regular expressions by using a regular expression engine called PCRE (Perl Compatible Regular Expressions). There is another module called *regex* which also supports regular expressions in Python. Even though this module offers some additional features when compared with the module *re*, we will use the module *re* in this tutorial for two reasons. First, *regex* is a third-party module whereas *re* is part of the Python standard library. Second, *regex* has an old and a new version, known respectively as version 0 and version 1 with major differences between the two. This makes

a study of the module *regex* even more difficult.

The module *re*

Python regular expressions simplify the task of pattern matching a lot by specifying a pattern that can match strings. The first thing you must do is import the module *re* with the command *import re*. Python does not support a new type for representing regular expressions; instead, strings are used for representing regular expressions. For this reason, a regular expression should be compiled into a pattern object, having methods for various operations like searching for patterns, performing string substitutions, etc. If you want to search for the word 'UNIX', the required regular expression is the word itself, i.e., *UNIX*. So, this string should be compiled with the function *compile()* of module *re*. The required command is *pat = re.compile('UNIX')*, where the object *pat* contains the compiled regular expression pattern object.

Optional flags of the function *compile()*

The function *compile()* has a lot of optional flags. Some of the important ones are DOTALL or S, IGNORECASE or I, LOCALE or L, MULTILINE or M, VERBOSE or X, and UNICODE or U. DOTALL changes the behaviour of the special symbol dot (.). With this flag enabled, even the new line character *\n* will be matched by the special symbol dot (.). IGNORECASE allows case insensitive search. LOCALE will enable a locale-aware search by considering the properties of the system being used. This allows users to perform searches based on the language preferences of their system. MULTILINE enables separate search on multiple lines in a single string. VERBOSE allows the creation of more readable regular expressions. UNICODE allows searches dependent on the Unicode character properties database.

Now, consider the regular expression with the flag IGNORECASE enabled, *pat = re.compile('UNIX', re.IGNORECASE)*. What are the strings that will be matched by this regular expression? Well, we are performing a case insensitive search on the word 'UNIX', so words like 'UNIX', 'Unix', 'unix' and even 'uNiX' will be matched by the given regular expression.

A Python script for regular expression processing

Consider the text file named *file1.txt* shown below to understand how a regular expression based pattern match works in Python.

```
unix is an operating system
Unix is an Operating System
UNIX IS an OPERATING SYSTEM
Linux is also an Operating System
```

Consider the Python script *run.py*, which reads a file name from the keyboard and opens it. The program then carries out